

## SKILL WEEK 9

### Task 1: List Active Jobs

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

To create a simple job table, display active jobs with their process IDs and states, and update the state after a job completes.

#### Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 5

struct Job {
    int id;
    pid_t pid;
    char state[20];
};

int main() {
    struct Job jobs[MAX_JOBS];
    int job_count = 0;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        sleep(5);
        exit(0);
    }

    jobs[job_count].id = 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state,
             sizeof(jobs[job_count].state),
             "Running");

    job_count++;
}
```

```

printf("myShell> jobs\n");
printf("\n--- Active Jobs ---\n");

for (int i = 0; i < job_count; i++) {
    printf("[%d] PID: %d State: %s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].state);
}

waitpid(pid, NULL, 0);

snprintf(jobs[0].state,
         sizeof(jobs[0].state),
         "Completed");

printf("\n--- Updated Job Status ---\n");

for (int i = 0; i < job_count; i++) {
    printf("[%d] PID: %d State: %s\n",
           jobs[i].id,
           jobs[i].pid,
           jobs[i].state);
}

return 0;
}

```

### Input

No keyboard input is required.

The program automatically creates a process and adds it to the job table.

### Output

```

amrutha@blue:~$ mkdir -p ~/SKILL_Week9
amrutha@blue:~$ cd ~/SKILL_Week9
amrutha@blue:~/SKILL_Week9$ nano jobs.c
amrutha@blue:~/SKILL_Week9$ gcc jobs.c -o jobs
amrutha@blue:~/SKILL_Week9$ ./jobs
myShell> jobs

--- Active Jobs ---
[1] PID: 3175 State: Running

--- Updated Job Status ---
[1] PID: 3175 State: Completed

```

## Observation

The active job was stored with a job ID, process ID, and state. The job state was updated after the child process completed.

## Result

Active job listing and job-state tracking were successfully implemented.

## Task 2: Foreground Job Control

**To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching**

To create a child process, transfer execution to the foreground, wait for the process to finish, and update its job state.

## Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Starting foreground job...\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process running in foreground. PID: %d\n",
            getpid());

        sleep(3);

        printf("Foreground job completed.\n");
        exit(0);
    }
}
```

```

printf("Job started with PID: %d\n", pid);
printf("Transferring execution to foreground...\n");

waitpid(pid, NULL, 0);

printf("Foreground job finished.\n");
printf("Job state updated to Completed.\n");

return 0;
}

```

### Input

No keyboard input is required.

### Output

```

amrutha@blue:~/SKILL_Week9$ nano foreground.c
amrutha@blue:~/SKILL_Week9$ gcc foreground.c -o foreground
amrutha@blue:~/SKILL_Week9$ ./foreground
Starting foreground job...
Job started with PID: 3216
Transferring execution to foreground...
Child process running in foreground. PID: 3216
Foreground job completed.
Foreground job finished.
Job state updated to Completed.

```

### Observation

A child process was created using `fork()`. The parent waited for the child using `waitpid()`, representing foreground execution until the target process completed.

### Result

Foreground job execution and completion tracking were successfully implemented.

## Task 3: Signal Delivery

### To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

To demonstrate signal delivery by handling signals such as `SIGINT` and `SIGTERM`.

### Program

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int signal) {
    if (signal == SIGINT)
        printf("\nSIGINT received. Process is still running.\n");
    else if (signal == SIGTERM)
        printf("\nSIGTERM received. Terminating process...\n");
}

int main() {
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT.\n");

    while (1) {
        printf("Process is running...\n");
        sleep(2);
    }

    return 0;
}

```

## Input

While the program is running, press:

Ctrl + C

This sends SIGINT.

## Output

```

amrutha@blue:~/SKILL_Week9$ nano signals.c
amrutha@blue:~/SKILL_Week9$ gcc signals.c -o signals
amrutha@blue:~/SKILL_Week9$ ./signals
Signal handling program started.
PID: 3272
Press Ctrl+C to send SIGINT.
Process is running...
Process is running...
Process is running...
Process is running...
^C
SIGINT received. Process is still running.
Process is running...
Process is running...
Process is running...
Process is running...
^Z
[1]+  Stopped                  ./signals

```

## Observation

The program received SIGINT from the terminal and executed the registered signal handler instead of terminating immediately.

## Result

Signal delivery and signal handling were successfully demonstrated.

## Task 4: Resume Stopped Jobs

**To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery**

To stop a child process, send SIGCONT to resume it, and update its job state.

## Program

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID: %d\n", getpid());

        for (int i = 1; i <= 5; i++) {
            printf("Child running: step %d\n", i);
            sleep(2);
        }

        printf("Child process completed.\n");
        exit(0);
    }

    printf("Job started with PID: %d\n", pid);

    sleep(2);

    printf("\nSending SIGSTOP...\n");
    kill(pid, SIGSTOP);

    sleep(2);
```

```

    sleep(2);

    printf("Job is now stopped.\n");

    printf("\nSending SIGCONT...\n");
    kill(pid, SIGCONT);

    printf("Job resumed successfully.\n");

    waitpid(pid, NULL, 0);

    printf("Job completed.\n");

    return 0;
}

```

### Input

No keyboard input is required.

The program internally sends:

SIGSTOP

and then:

SIGCONT

### Output

```

amrutha@blue:~/SKILL_Week9$ nano resume.c
amrutha@blue:~/SKILL_Week9$ gcc resume.c -o resume
amrutha@blue:~/SKILL_Week9$ ./resume
Job started with PID: 3331
Child process started. PID: 3331
Child running: step 1

Sending SIGSTOP...
Job is now stopped.

Sending SIGCONT...
Job resumed successfully.
Child running: step 2
Child running: step 3
Child running: step 4
Child running: step 5
Child process completed.
Job completed.

```

**Observation**

The child process was stopped using SIGSTOP and later resumed using SIGCONT. The parent waited for the resumed process to complete.

**Result**

Stopped-job recovery using SIGCONT and process-state management were successfully implemented.